claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_3512746c-54e\agent-a5c3a7b65f4f0536b.jsonl`
- SHA-256: `21dcd91055d4add144734d52f4bd94a4af83d6287f221149d6e2ca69602647fa`
- Source modified: `2026-07-06T09:09:49+00:00`
- Imported at: `2026-07-08T16:00:05+00:00`
- project: `wf_3512746c-54e`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T09:09:16.747Z)

```
Reviewing GitHub PR #39 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-e-
selective-scan -> main, one commit db87cc0).
It implements ADR 0009 Track E: selective source scanning ? /watch, /unwatch, /channels,
/watchlist commands +
an ingestion gate in telethon_client.py that skips unwatched sources. Changed files ONLY:
command_service.py (new /watch etc + _resolve_source), commands.py (re-export patterns), db.py
(set_source_watched, list_account_sources), telethon_client.py (the gate),
tests/test_command_service.py (10 tests).
Diff +254/-3. READ docs/adr/0009-selective-channel-scanning.md ? it is the spec this must
conform to.

KEY ADR 0009 REQUIREMENTS (quote/verify against the code):
- Line 37-38: "The default is locked as opt-in: DEFAULT 0, so a freshly-onboarded user scans
nothing until they opt in."
- Line 44: "...return early if watched is false, before any dedup or filter work."
- Line 130-131: "New channels discovered after the migration default to watched = 0 and must be
explicitly watched."
- Line 48-54 (discoverability): lazy-populated rows should be created watched=0 so they appear
in /channels for opt-in;
  /channels should also upsert candidates from client.get_dialogs().
- Line 64: "/watch <#name|id>" ? resolve by channel name/username OR id.

Ground rules:
- READ actual code (Read/Grep), cite file:line. You MAY run repros with
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe (build schema via db._apply_schema
  on aiosqlite ":memory:"; seed via db.upsert_account + db.upsert_source; drive
command_service.CommandService().handle()).
  NOTE: real telegram sources have a NUMERIC provider_source_id (str(channel_id) e.g.
'-1001234567890'); upsert_channel
  creates them with watched=True. The on_new_message gate is inside a '# pragma: no cover'
function.
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check + format clean; pytest
492 passed @ 97.10% (floor 80%).
- Report ONLY defects in THIS PR's diff. Deviation from the ACCEPTED ADR 0009 is a high-value
finding. No praise.
  Concrete failure scenario per finding, ranked most-severe first, honest severity.
- Severity: blocker (feature broken / data loss on the live path), high (real bug or ADR-
contract violation users hit),
  medium (edge/latent/half-broken feature), low (hygiene/UX/discoverability).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read the cited code AND
docs/adr/0009-selective-channel-scanning.md,
trace real control flow, run a repro with
```

```
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful. CONFIRMED only if the
defect genuinely ships in THIS PR's diff
and (for ADR-conformance) actually deviates from the accepted ADR. REFUTED if the code conforms
/ trigger can't occur /
pre-existing. PLAUSIBLE if unsettled. Corrected severity.
FINDING (tests_ux): format_commands() omits /watch, /unwatch, /channels, /watchlist --
undiscoverable via /show commands
severity low | src/tg_video_filter/command_service.py:129
summary: The four new Track E commands are not listed in format_commands(), so users cannot
discover them through the /show commands help, and the existing
test_format_commands_includes_all_sections does not assert them.
scenario: A user runs /show commands to learn what they can do. The help text (format_commands,
command_service.py:129-160) lists filter config, exclusions, backfill, and help -- but nothing
about watching sources. The selective-scanning feature is invisible; a user who doesn't already
know the exact syntax cannot find /watch or /channels. Repro confirms all four commands are
absent from the help output.
evidence: Repro: for each of /watch, /unwatch, /channels, /watchlist, `cmd in format_commands()`
returns False. test_format_commands_includes_all_sections (test line 231-236) only checks /show
config, /set size, /set exclude, /backfill -- it would not catch the omission.
rec: Add a 'Source scanning' section to format_commands() documenting /channels, /watch <id>,
/unwatch <id>, /watchlist, and extend the format_commands test to assert their presence.
```

2. assistant (2026-07-06T09:09:21.244Z)

I'll verify this finding by reading the actual code and the ADR spec.

3. user (2026-07-06T09:09:27.267Z)

```
110     def format_config(cfg: FilterConfig) -> str:
111         """Format a ``FilterConfig`` as a readable message (Telegram markdown)."""
112         lines = [
113             "? **Current Filter Configuration**",
114             "",
115             f"??  **Duration:** {_format_duration(cfg.min_duration_s)} -
{_format_duration(cfg.max_duration_s)}",
116             f"? **Min resolution:** {'? ' + str(cfg.target_height) + 'p' if
cfg.target_height else 'disabled'}",
117             f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
118             f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
119             f"? **Delivery target:** `{cfg.delivery_target}`",
120             "",
121             "? **Content exclusions:**",
122             f"   GIF/animated: {'? excluded' if cfg.exclude_gif else '? allowed'}",
123             f"   Round videos: {'? excluded' if cfg.exclude_round else '? allowed'}",
124             f"   Silent (no audio): {'? excluded' if cfg.exclude_nosound else '? allowed'}",
125         ]
126         return "\n".join(lines)
127
128
129     def format_commands() -> str:
130         """Return a help message listing all available commands."""
131         return "\n".join(
132             [
133                 "? **Available Commands**",
134                 "",
135                 "? **Filter config**",
136                 "`/show config` ? Show current filter settings",
137                 "`/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
138                 "   Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing
`B`",
139                 "`/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
140                 "   or `/set max length <N><unit>` ? same as above",
141                 "   Units: `s` (default), `m`, `h` ? case-insensitive",
142                 "`/set min length <N><unit>` ? Set min video duration (e.g. `/set min length
5s`)",
```

```
143                 "   Units: `s` (default), `m`, `h` ? case-insensitive",
144                 "`/set min resolution <N>p` ? Set min video height (e.g. `/set min
resolution 720p`)",
145                 "   or `/set min res <N>p` ? same as above",
146                 "",
147                 "? **Content exclusions** (all off by default)",
148                 "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
149                 "`/set exclude round on|off` ? Exclude circular video messages",
150                 "`/set exclude nosound on|off` ? Exclude videos without audio\n"
151                 "   Note: GIFs are usually silent; `exclude nosound` will catch most GIFs
too.",
152                 "",
153                 "? **Backfill**",
154                 "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history
(default 90d)",
155                 "`/stop` ? Cancel running backfill",
156                 "",
157                 "\u2139\ufe0f  **Help**",
158                 "`/show commands` ? Show this help message",
159             ]
160         )
161
162
163     # --- command result -----------------------------------------------------------
164
165
166     @dataclass
167     class CommandResult:
168         """The output of executing a command ? provider-independent.
169
170         The adapter (Telethon or Bot API) uses ``reply_text`` to send the response
171         and ``config_changed`` to decide whether to hot-reload the filter engine.
172         """
173
174         reply_text: str = ""
175         config_changed: bool = False
176
177
178     # --- command service ----------------------------------------------------------
179
180
181     class CommandService:
182         """Provider-independent command execution.
183
184         Parses a command string against an account's current config and DB,
185         returning a ``CommandResult``. No Telethon objects involved ? pure string
186         in, effects out. The Telethon adapter (``commands.py``) handles the
187         transport (checking message.out, sending the reply); the Bot API adapter
188         (future) calls ``handle()`` with the ``from.id`` from the update.
189
```

4. user (2026-07-06T09:09:27.656Z)

```
1       # ADR 0009: Selective channel scanning and multiplexed routing
2
3       - **Status**: Accepted
4       - **Date**: 2026-07-04
5       - **Decision owner**: project owner
6
7       ## Context
8
9       Today the userbot scans **every** channel/group the account is a member of.
10      `manager.py` registers `events.NewMessage(incoming=True)` with no `chats=`
11      filter, and `on_new_message` processes every video regardless of source. The
12      `channels` table exists but is populated lazily as a *record* of seen channels;
```

it is not used as a filter.

The product vision is a **multiplexing video re-distribution system**:

- A small set of high-quality **source** channels (e.g. spiritual, classical
  music) are scanned for incoming videos.
- Matches are routed to themed **destination** broadcast channels (e.g. a
  Reggae channel, a Rap channel) that subscribers opt into.
- A user therefore subscribes to *outputs* by topic, not to *inputs* by source.

This requires two capabilities the current architecture does not have:

1. **Selective scanning** ? choose which source channels are watched, rather
   than ingesting all of them.
2. **Per-source ? per-destination routing** ? a match from source channel X is
   delivered to a specific destination channel Y, instead of the single global
   `delivery_target` (ADR 0005).

Decision

Add three additive, independently-shippable capabilities:

1. Selective scanning (watched-flag)

Add a `watched` flag to the `channels` table. The default is locked as
opt-in: `DEFAULT 0`, so a freshly-onboarded user scans nothing until they
explicitly `/watch` a channel. This matches the multiplexing product shape
(sources are a *curated* set) and avoids blindly re-ingesting every chat the
account is a member of.

Gate `on_new_message` at the top: after computing `channel_id`, look up the
row and `return` early if `watched` is false, before any dedup or filter work.

This is a small migration plus ~5 lines in `telethon_client.py`.

Discoverability note. With `DEFAULT 0` plus the current lazy population
(a `channels` row is only inserted when a message is first seen), opt-in is
necessary but not sufficient for discoverability: a channel the account has
never received a message from has no row to flag. The `/channels` command
(step 2 below) therefore upserts candidate rows from `client.get_dialogs()`
so users can browse and watch channels they have not yet received traffic
from. The flag default and the enumeration flow are orthogonal ? both are
required.

2. Channel enumeration and management commands

New Saved Messages commands (and, via ADR 0008, bot commands) building on the
existing `commands.py` pattern:

- `/channels` ? list the user's channels (via `client.get_dialogs()`, already
  used in `backfill.py`) with their watched state.
- `/watch <#name|id>` ? set `watched=1` for a source channel.
- `/unwatch <#name|id>` ? set `watched=0`.
- `/watchlist` ? list only watched sources.

3. Per-source ? per-destination routing rules

Introduce a `routing_rules` table (the canonical name; issue #25's `routes`
refers to the same concept and should be aligned to `routing_rules`), e.g.:

```
routing_rules (
  user_id            INTEGER NOT NULL REFERENCES users(id),
  source_channel_id  INTEGER NOT NULL,
  dest_channel_id    INTEGER NOT NULL,   -- a broadcast channel / @username
```

```
  filter_config_id   INTEGER,            -- optional per-route filter override
  PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
)
```

On a match, look up applicable routing rules by `source_channel_id` and deliver to each matching destination. The delivery layer (`delivery.py`) already accepts a `target` per call, so this generalises the existing single `delivery_target` to a per-match lookup. Users with no routing rules fall back to the current global `delivery_target` behaviour (ADR 0005), preserving backwards compatibility.

Consequences

Positive
- Enables the multiplexing product shape: scan curated sources, publish to topical broadcast channels, let subscribers pick outputs by interest.
- Each capability is additive and independently deployable; none disturbs the existing FORWARD/LINK/BOTH paths or the metadata-first ingestion (ADR 0003).
- Dedup (ADR 0006) still holds: Tier 1 (`document_id`) keys the *original* file, so a routed re-upload is not re-processed.
- Channel enumeration reuses `client.get_dialogs()` already present in `backfill.py`; no new Telethon capability is required.

Negative
- New schema (`channels.watched`, `routing_rules`) with migrations.
- **Routing loop invariant.** With multiplexed routing, the userbot account may be a member of a *destination* broadcast channel and re-see its own output, producing an infinite forward loop. The contract is: **a channel is either a source or a destination, never both.** This is enforced in `on_new_message` by excluding any channel that is a configured destination (`source_channel_id IN (SELECT dest_channel_id FROM routing_rules WHERE user_id = ?)`) before the watched-flag check. Issue #25 Track E implements this; the invariant is stated here so the handler has a contract to code against.
- Per-route filter overrides add config complexity; v1 may ship with global filters only and a single implicit route (`source ? global target`), then layer explicit per-route rules later.

Backwards compatibility

Adding `channels.watched` with `DEFAULT 0` would silently stop every existing install from receiving matches until the user re-watches their channels ? a behaviour change disguised as a schema change. To preserve the current "ingest everything" semantics for existing installs, the migration backfills:

```sql
ALTER TABLE channels ADD COLUMN watched INTEGER NOT NULL DEFAULT 0;
UPDATE channels SET watched = 1 WHERE seen_first_at IS NOT NULL;
```

i.e. any channel the install has *already* seen traffic from is implicitly watched, so pre-migration users observe no change. New channels discovered after the migration default to `watched = 0` and must be explicitly watched ? which is the intended new UX. This mirrors the "old rows behave exactly as before" pattern established by ADR 0005.

Rollout order

1. **Selective scanning** (`channels.watched` + `/watch`, `/unwatch`, `/channels`). Smallest unit; immediately useful even without routing.
2. **Named destination channels** ? extend `/set delivery_target` to allow switching among a user-defined set, not just one global value.
3. **Per-source ? per-destination routing rules** (`routing_rules` table + `/route <source> <dest>` command). The full multiplexing vision.

```
143
144     Each step is shippable and useful on its own; step 3 is not required to
145     deliver value from steps 1?2.
146
147     ## Alternatives considered
148
149     - **Server-side `chats=` filter at handler registration:** rejected ? the
150       watched set is per-user and mutable at runtime; re-registering Telethon
151       handlers on every `/watch` change is brittle. An in-handler lookup against
152       the `channels` table is simpler and hot-swappable.
153     - **Content-based classification (auto-detect genre):** explicitly out of
154       scope for v1. Routing is user-defined by source channel; automatic topic
155       detection could be a future enhancement but is not required to deliver the
156       multiplexing UX.
157
158     ## References
159
160     - ADR 0001 (userbot ingestion) ? the capability this selectively constrains.
161     - ADR 0005 (per-user delivery target) ? the single-target delivery this
162       generalises into per-route targets.
163     - ADR 0006 (content dedup) ? still applies; routed re-uploads are keyed by
164       the original `document_id`.
165     - ADR 0008 (bot API onboarding) ? the front-door that exposes `/watch`,
166       `/unwatch`, and routing commands to public users.
167
```

5. assistant (2026-07-06T09:09:31.027Z)

Now I've read the ADR. Let me verify the finding by checking format_commands() output and the test.

6. user (2026-07-06T09:09:32.652Z)

```
43:_WATCH_RE = re.compile(r"^/watch\s+(.+)$", re.IGNORECASE)
44:_UNWATCH_RE = re.compile(r"^/unwatch\s+(.+)$", re.IGNORECASE)
45:_CHANNELS_RE = re.compile(r"^/channels$", re.IGNORECASE)
46:_WATCHLIST_RE = re.compile(r"^/watchlist$", re.IGNORECASE)
318:        m = _WATCH_RE.match(text)
320:            return await self._handle_watch(conn, user_id, m.group(1).strip(), current_cfg)
322:        m = _UNWATCH_RE.match(text)
324:            return await self._handle_unwatch(conn, user_id, m.group(1).strip(),
current_cfg)
326:        if _CHANNELS_RE.match(text):
327:            return await self._handle_channels(conn, user_id)
329:        if _WATCHLIST_RE.match(text):
330:            return await self._handle_watchlist(conn, user_id)
341:    async def _handle_watch(
344:        """Parse ``/watch <@username|channel_id>`` and toggle ``watched=1``."""
348:                f"? Source `{target}` not found. Use `/channels` to list "
353:        await db.set_source_watched(conn, user_id, src.id, True)  # type: ignore[arg-type]
355:        return CommandResult(f"? Now watching **{label}** (`{src.provider_source_id}`).")
357:    async def _handle_unwatch(
360:        """Parse ``/unwatch <@username|channel_id>`` and toggle ``watched=0``."""
364:                f"? Source `{target}` not found. Use `/channels` to list sources."
366:        await db.set_source_watched(conn, user_id, src.id, False)  # type: ignore[arg-type]
368:        return CommandResult(f"? Stopped watching **{label}**
(`{src.provider_source_id}`).")
370:    async def _handle_channels(self, conn: aiosqlite.Connection, user_id: int) ->
CommandResult:
371:        """List all known sources with watch status."""
377:                "channel. Run `/backfill` to discover historical channels, "
382:            icon = "?" if s.watched else "?"
387:    async def _handle_watchlist(self, conn: aiosqlite.Connection, user_id: int) ->
CommandResult:
388:        """List only watched sources."""
```

```
389:        sources = await db.list_account_sources(conn, user_id, watched_only=True)
392:                "? **No watched sources.**\n\n"
393:                "Use `/channels` to see available sources, then `/watch <id>` "
396:        lines = ["? **Watched sources**", ""]
```

7. assistant (2026-07-06T09:09:38.154Z)

The finding is confirmed by reading the code. `format_commands()` (lines 129-160) has no mention of `/watch`, `/unwatch`, `/channels`, or `/watchlist`. All four are implemented and dispatched (lines 318-330) but absent from help. Let me confirm the test assertion scope.

8. user (2026-07-06T09:09:38.780Z)

```
225         assert result.config_changed is False
226
227
228     # --- format_commands -----------------------------------------------------
229
230
231     def test_format_commands_includes_all_sections() -> None:
232         text = format_commands()
233         assert "/show config" in text
234         assert "/set size" in text
235         assert "/set exclude" in text
236         assert "/backfill" in text
237
238
239     # --- /channels + /watch + /unwatch + /watchlist (ADR 0009, Track E) ----------
240
241
242     async def _seed_source(
243         conn: aiosqlite.Connection,
244         account_id: int,
```

9. assistant (2026-07-06T09:09:44.099Z)

Confirmed by direct code reading. The `format_commands()` help text (command_service.py:129-160) lists filter config, exclusions, backfill, and help but omits all four Track E commands. The test at line 231-236 only asserts the pre-existing four command strings, so it would not catch the omission.

This is a genuine discoverability gap shipped in this PR's diff. The ADR does not mandate a help entry, but the feature adds a user-facing `/show commands`-style help function and fails to register the new commands there. Severity is correctly low ? it is a UX/discoverability hygiene issue, not a functional bug. The commands themselves work when invoked with correct syntax.

10. user (2026-07-06T09:09:49.801Z)

Structured output provided successfully